

AI datacenter room planner

Rev 0.1 · 2026-07-25

Generated from Markdown — edit the .md source and regenerate (python3 docs/md2pdf.py)

Contents

1	Run	3
2	What you control	4
2.1	Cooling is load-bearing, not cosmetic	4
2.2	Power is modeled end to end	5
3	The algorithms	6
3.1	Traffic matrix (<code>js/graph.js</code>)	6
3.2	Partitioning (<code>js/partition.js</code>)	7
3.3	Placement (<code>js/placement.js</code>)	7
3.4	Routing (<code>js/pathways.js</code>)	7
3.5	Bundling (<code>js/pathways.js</code> → <code>steiner</code>)	8
3.6	Media and cost	8
4	Output: the source of truth	9
4.1	Label conventions	9
5	Validation	11
6	Determinism	12
7	Example	13
8	What this is not	14

Interactive design-time simulator for an AI hall. Set the room size,
pick air or water cooling, add and remove racks, choose the server
layout and the switching architecture, place the power plant — and get
a **rack and cable labeling YAML source of truth** out the other
end.

Where `demo/` simulates *bringing up* a fixed
64× B200 cluster, this tool decides *what to build in the first*
place.

Specification: `SPEC.md` — requirements,
architecture, normative output schema and label grammar, algorithm
specification, the full validation rule catalog, and the
verification/traceability plan. Read that if you are changing behaviour;
read this if you are using the tool.

1 Run

```
python3 -m http.server 8777 --directory planner  
# → http://127.0.0.1:8777
```

index.html also opens directly from disk — there is no build step
and no dependency beyond a browser.

Headless, for CI or scripting:

```
node planner/tools/plan.js --report          # summary to stdout  
node planner/tools/plan.js -o hall-a.yml     # write the SoT  
node planner/tools/plan.js --set cooling.mode=air --set fabric.arch=tor --report  
node planner/tools/plan.js my-design.json -o hall-a.yml  
  
python planner/tools/validate_design.py hall-a.yml # independent re-check
```

plan.js exits non-zero when validation finds an error, so a bad design
fails a pipeline instead of shipping.

2 What you control

Control	Effect
Room width / depth / height	Recomputes the row grid: how many back-to-back pairs fit, how many slots per row, how many rack positions exist at all
Aisles cold / hot	Row pitch, and therefore every cable length in the hall
Cooling air ↔ water	Sets the per-rack kW ceiling, gates which rack layouts are legal, and decides whether you get CRAHs, rear-door HXs, or in-row CDUs
Racks add / remove / servers-per-rack	The fleet. Drag a rack on the floor plan to pin it; everything unpinned is solver-placed
Rack layout	Which server SKU fills the rack — XE9680, HGX B200/B300, GB200 NVL72 trays, CPU, JBOF, mgmt
Switching architecture	Rail-optimized (middle-of-row), top-of-rack, or end-of-row — plus oversubscription and 2 vs 3 tiers
Power	Utility entrances (A/B), UPS model and redundancy, RPP panels vs overhead busway, rack PDU rating, optional per-PSU cords
Workload TP / PP / DP	The traffic matrix that drives partitioning and placement
Optimizer	Which solvers run and how hard

2.1 Cooling is load-bearing, not cosmetic

Selecting **air** caps racks at ~40 kW and makes liquid-only SKUs

(GB200 NVL72, direct-liquid B300) illegal — the planner reports that instead of quietly swapping your hardware:

```
ERROR [cooling.rack_cap] GPU-01 draws 100.95 kW but contained air tops out at 40 kW/rack
ERROR [cooling.layout] GPU-01: GPU · HGX B300 4U ×8 (direct liquid) has no air-cooled variant
```

water/dlc adds in-row CDUs that consume real floor positions, and a supply/return pair per rack manifold. **water/rdhx** adds depth to every row instead, which changes the row pitch and every cable length with it.

2.2 Power is modeled end to end

utility entrance → UPS → RPP or overhead busway → rack PDU → PSU cords

(A / B) (A/B) (A/B) (A/B)

Two rules drive the sizing:

1. **Dual-corded means each side survives alone.** A and B normally share the load, but each is sized for 100% — so a 101 kW rack needs three 60 A PDUs *per side*, not three in total.
2. **Continuous load is derated to 80%.** A 60 A 415 V rack PDU is a 34.5 kW rack PDU.

Panels are sited in a second pass at the centroid of the racks they picked up, and any panel nothing landed on is dropped rather than sold to you. Switching from RPP to busway visibly relieves tray congestion, because the whips stop riding the cable tray.

3 The algorithms

There is no single algorithm that produces a cable plan. This is the standard hybrid, implemented small enough to run in a browser tab:

```

Logical topology (GPU → NIC → leaf → spine → core)
    |
    ▼   traffic matrix from the parallelism plan (TP / PP / DP)

Graph partitioning ..... multilevel coarsening + Kernighan-Lin/Fiduccia-Mattheyses
    |
    |                                     → which server lives in which rack
    ▼

Rack placement ..... Quadratic Assignment Problem, greedy seed +
    |
    |                                     simulated annealing (or an order-crossover GA)
    ▼
    |                                     → where each rack sits on the floor

Pathway routing ..... A* over a tray graph, congestion- and bend-aware,
    |
    |                                     Yen's K-shortest for A/B power diversity
    ▼

Bundling ..... Steiner tree (shortest-path heuristic) per spine
    |
    |                                     → trunk runs instead of N independent cables
    ▼

Cable schedule ..... routed length → medium → cost → labels → YAML

```

3.1 Traffic matrix (js/graph.js)

Demand comes from the parallelism plan, not from cable counts:

Edge	Weight	Why
tensor-parallel peers	400 GB/s	all-to-all every layer — must stay in-rack
pipeline stage boundary	40 GB/s	activations, point to point
data-parallel ring	120 GB/s	gradient / KV all-reduce
background	×0.02	storage, checkpoint, man- agement

A job needs $TP \times PP \times DP$ GPUs; a larger fleet runs several instances side by side rather than one impossibly wide job.

3.2 Partitioning (js/partition.js)

Heavy-edge matching coarsens the demand graph, a capacity-aware greedy growth seeds the parts, and FM refinement with **best-prefix**

rollback runs at every level on the way back up. Because racks are usually filled exactly, there is no slack to move a vertex into — so refinement also does KL-style **swaps**, with the classic gain

$$D_v + D_u - 2 \cdot c(v, u).$$

The objective is edge cut: the GB/s that has to leave a rack, which is exactly the traffic that must cross the leaf tier.

With TP=8 on 8-GPU nodes a tensor-parallel group *is* one server, so there is nothing to gain and the report honestly says **-0%**. Set TP=16 (two nodes per group) and the same solver cuts inter-rack demand by **77%**.

3.3 Placement (js/placement.js)

$$\text{minimize } \sum_{\{i,k\}} \sum_{\{j,l\}} F_{ik} \cdot D_{jl} \cdot X_{ij} \cdot X_{kl}$$

F is multi-objective: a weighted mix of the **traffic** matrix (from partitioning) and the **cable-count** matrix (from the fabric plan). The second term matters because $\sum F_{\text{cable}} \cdot D$ is total cable length — so the same solver that keeps NCCL traffic local also minimises the cable order. The slider in the UI moves between the two. **D** is Manhattan with a penalty for crossing rows, because cables run along the row and cross at an aisle.

QAP is NP-hard, so: greedy constructive seed → simulated annealing over swaps and relocations, every candidate evaluated incrementally in $O(n)$.

A genetic option (order crossover, tournament selection, elitism) is available and is always polished with a short anneal.

Nothing hard-codes “put the network rack in the middle of the row” — give the network rack a large flow term to every compute rack and the solver puts it there on its own, because that is what minimises $\sum F \cdot D$.

3.4 Routing (js/pathways.js)

Cables do not fly between racks. The planner builds a tray graph — a spine over every aisle, cross-aisle ladders at the ends and every 6 columns, a drop into each rack — on **three separate tiers**:

data, power, and coolant.

```
w = length · (1 + congestion_weight · fill)  ← tray utilization
      + bend_penalty if the run changes axis  ← bends
```

so the cheapest route is often not the shortest one. A* runs over (node, direction) states so bends can be charged. Fill is committed after each cable, which makes routing order-dependent in the same way a real install is: the first trunks get the good pathway.

The **A/B power feeds** are routed for real diversity: the A feed goes first, then Yen's K-shortest generates candidates for B and the one sharing the fewest tray segments with A wins. If the room offers no disjoint path, that is reported rather than assumed away.

3.5 Bundling (js/pathways.js → steiner)

For each spine, its leaf connections are solved as a Steiner tree over the tray graph (shortest-path heuristic: repeatedly splice in the nearest unconnected terminal, with existing tree segments priced at 5% so reuse is preferred). That turns N independent spine→leaf cables into shared trunk runs, and every cable carries its **bundle** id into the schedule.

3.6 Media and cost

The **routed** length — rack rise + tray run + drop + slack — selects the cheapest medium that still reaches:

	reach	installed cost/link
DAC 400G	3 m	\$260
AEC 400G	5 m	\$690
AOC 400G	30 m	\$1,450
DR4 + MPO SMF	500 m	\$2,600

This is why the switching architecture matters so much: top-of-rack keeps host cables inside the rack on DAC, rail-optimized pushes all 512 of them onto AOC.

4 Output: the source of truth

The YAML export is the deliverable. Sections:

Key	Contents
room	Dimensions, aisles, row grid, position count, pathway tiers
cooling	Mode, per-rack ceiling, capacity, ΔT , flow, every CRAH/CDU/RDHx/manifold with coordinates
power	Entrances, UPS units, RPP/busway units, every rack PDU — with feed, rating, load and coordinates
fabric	Architecture, port split, achieved oversubscription, switch counts
racks	Position, frame, totals, and a full U-by-U elevation
devices	Flat device index (name, model, rack, U, kW, ports)
cables	Every cable: label, class, medium, routed length, bends, bundle, both endpoints
media	Reach and cost of each medium used, so the file re-checks itself
bundles	Trunk groups with shared pathway length
labels.conventions	What each label pattern means
optimization	What each solver achieved vs its baseline
validation	Every finding, with severity and code

4.1 Label conventions

Extends the family already used in

`docs/cabling.md`:

Pattern	Meaning
R{rail}-{server}	Host NIC → rail leaf (rail-optimized)
H{nic}-{server}	Host NIC → ToR / EoR leaf
L{leaf}S{spine}-U{n}	Leaf → spine uplink
S{spine}X{super}-U{n}	Spine → super-spine uplink
PEER-{switch}-{n}	MLAG peer-link
00B-{device} / MGMT-{device} / MA1-{device}	BMC / mgmt0 / switch Management1
PE-{feed}-{ups}	Utility entrance → UPS
PF-{feed}-{unit}	UPS → RPP or busway

Pattern	Meaning
PW- {feed}{n} - {rack} / PT-...	RPP breaker → rack PDU whip / busway tap
PC- {feed} - {device} -P {n}	Rack PDU outlet → device PSU
CW- {S\ R} - {rack}	Rack manifold → CDU (secondary loop)
CF- {S\ R} - {unit}	CDU / CRAH → facility loop (primary)

Print both ends. A single leaf-per-rail keeps the familiar **L0S1-U1** form; when a rail needs a second leaf the tag grows a group suffix (**L0g2S1-U1**) so labels stay unique.

5 Validation

Two independent checkers, on purpose:

- `js/validate.js` runs inside the build and populates the findings panel.
- `tools/validate_design.py` re-derives the checks from the YAML alone and **disagrees loudly** if the document's own verdict does not match. A bug in the generator shows up as a disagreement rather than as two copies of the same mistake.

Checked: rack RU capacity and elevation overlap, floor loading, per-rack cooling ceiling and layout legality, cooling capacity, entrance / UPS / RPP / rack-PDU capacity with derate and single-feed survival, switch port budgets, rail identity, duplicate labels, port double-booking, media reach, tray fill, aisle clearance, A/B pathway diversity, and TP groups split across racks.

6 Determinism

Every solver runs off a seeded PRNG. The same design always produces byte-identical YAML — a source of truth that reshuffles on reload is not a source of truth. Change the seed to explore a different local optimum.

7 Example

`examples/pod-small.json` is a compact
4-rack pod; `examples/pod-small.yml` is
its export. Regenerate:

```
node planner/tools/plan.js planner/examples/pod-small.json -o planner/examples/pod-small.yml  
python planner/tools/validate_design.py planner/examples/pod-small.yml
```

8 What this is not

Honest limits, so nobody mistakes the output for a stamped drawing:

- **Heuristics, not proofs.** Partitioning, QAP, and Steiner bundling are all heuristic. The report always shows the achieved objective against a baseline so you can see what the search actually bought.
- **No CFD, no fluid pressure model.** Cooling is a kW and ΔT budget with flow derived from $Q = \dot{m} \cdot c_p \cdot \Delta T$; it will not tell you about a recirculation problem or a pump curve.
- **No electrical fault study.** No short-circuit, selective coordination, arc flash, or harmonics. Sizing is capacity and derate only.
- **Catalog figures are planning figures.** Nameplate or typical-sustained numbers good enough to size a room and a cable order — not a substitute for a vendor site-prep guide at procurement.
- **Codes are not encoded.** Power/data separation is modeled as distinct pathway tiers, but NEC/IEC clearances, seismic bracing, and fire suppression are out of scope. A licensed engineer still signs the drawing.